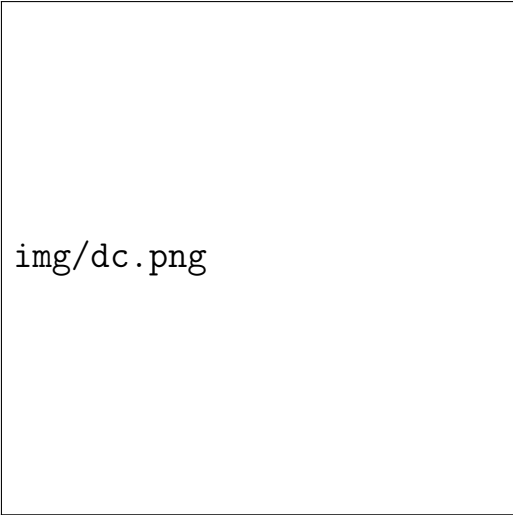




img/dc.png

Инструменты обучения нейронных сетей

Высшая Школа Цифровой Культуры
Университет ИТМО
dc@itmo.ru

Содержание

1	Инструменты обучения нейронных сетей	2
1.1	Введение	2
1.2	Autograd	2
1.3	Tensorflow	4
1.4	Keras	7
1.5	Hyperparams	7
1.6	Under- and Over-fitting	7
2	Оптимизаторы в НС	8
2.1	Введение	8
2.2	SGD и скорость обучения	8
2.3	Моментум (Momentum), импульс	10
2.4	Метод Ньютона	10
2.5	Adagrad и Adadelata	11
2.6	RMSProp	12
2.7	Adam	12
2.8	Практические советы	13

1 Инструменты обучения нейронных сетей

1.1 Введение

Здравствуйтесь, уважаемые слушатели. В этой лекции мы приступим к изучению библиотек для тренировки нейронных сетей. В предыдущей лекции мы уже познакомились с бэкпропом – эффективным способом вычисления градиента, используя проход по графу вычислений. Впрочем, мы ограничились рассмотрением лишь скалярных функций. Конечно, в реальных задачах мы работаем с матрицами, но идейно сама схема остается прежней. Исключения составляют некоторые формулы для вычисления матричных производных. Оно и понятно, матричное умножение устроено несколько сложнее, чем обычное, привычное нам умножение двух скаляров. Мы не будем детально останавливаться на том, как меняются формулы и (или) выражения для тех или иных производных, а сразу окунемся в инструменты, которые помогут нам автоматически дифференцировать то, что мы хотим. Поехали.

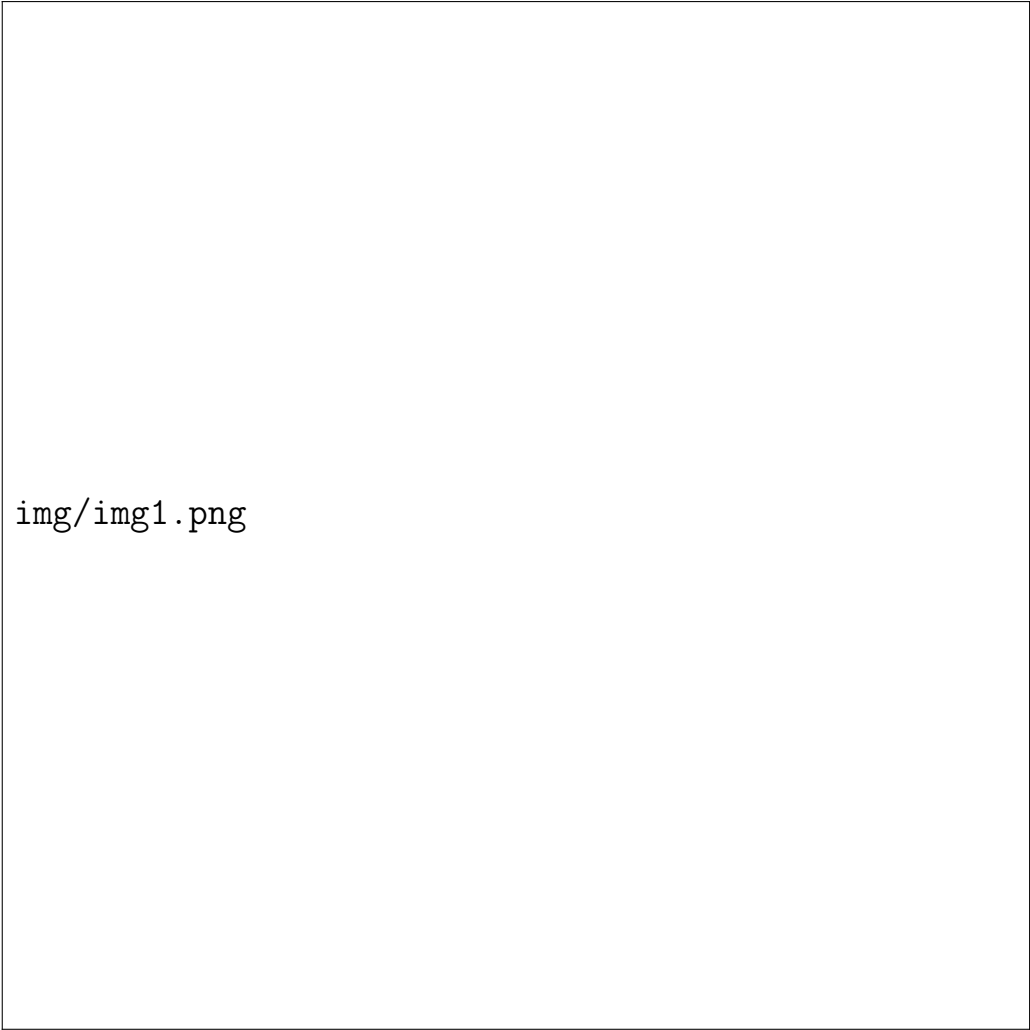
1.2 Autograd

Итак, в предыдущей лекции мы с вами уже выяснили, что такое аппарат нейронных сетей, и какая математика за ним кроется. Теперь давайте разберемся, какие в настоящее время существуют средства для облегчения процесса разработки моделей, как они работают и как ими пользоваться.

Как мы помним, с точки зрения математики, нейронная сеть – это комбинацией различных слоев, каждый из которых представлен матрицей весов и правилами его применения. При таком подходе функцию активации мы с вами тоже будем рассматривать как слой, просто с другими весами (некоторые функции активации, такие, как `ParametrizedReLU`, имеют обучаемые параметры) и другими правилами применения этого слоя и подсчета градиента. Данные слои могут комбинироваться различными способами, образуя структуру сети. Структура сети, на самом деле – синоним графа вычислений. Пример вы можете видеть на рисунке 1.2

Т.к. многие слои – строительные блоки нейронных сетей – стандартизованы и реализуются одинаковым образом, были созданы специальные библиотеки, которые за внешним интерфейсом, удобным для программиста, скрывают монотонный код описания прямого прохода и обратного распространения ошибки через данный граф. Названия самых популярных библиотек вы можете увидеть на рисунке 1.2.

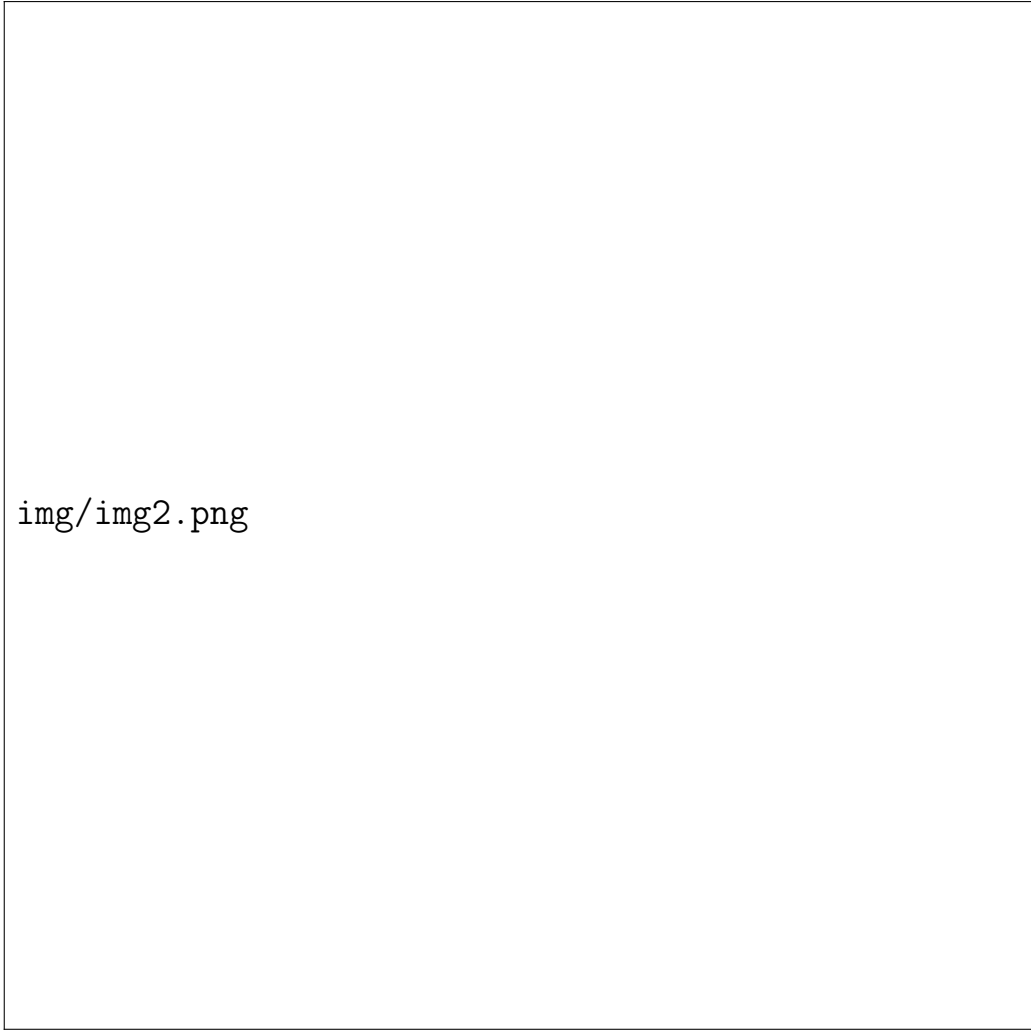
Давайте рассмотрим две самые популярные библиотеки немного поближе – это TensorFlow от команды Google Brain и PyTorch, разрабатываемая Facebook AI Research lab. Обе эти библиотеки дают возможность построить необходимый вам граф вычислений из различных слоев, добавлять на эти



img/img1.png

слои ограничения, изменять параметры и функции активации, использовать различные оптимизаторы, а также производить обучение моделей на графических или иных специализированных ускорителях. Кроме того, одной из важнейших причин использования таких библиотек является наличие т.н. autograd'a – движка автоматизированных вычислений производных, который следит за тем, как слои и компоненты соединяются друг с другом и реализует прямое и обратное распространение ошибки. Это обеспечивается тем, что для каждой операции на графе вычислений, построенном с помощью данных библиотек, хранится информация о том, как вычислять градиент, позволяя в дальнейшем autograd'у произвести необходимые операции. Таким образом инженер избавлен от необходимости самостоятельно писать однотипные функции вычисления градиента и может сосредоточиться на более высокоуровневых вещах, таких как архитектура сети или используемые гиперпараметры.

В рамках данного курса мы будем использовать библиотеку TensorFlow, о которой сейчас и поговорим.



img/img2.png

1.3 Tensorflow

TensorFlow – это открытая программная библиотека для машинного обучения, которая была разработана для построения и тренировки искусственных нейронных сетей. Данная библиотека была разработана компанией Google и в дальнейшем выложена в открытый доступ, и в настоящее время является одной из самых популярных библиотек для построения и обучения нейронных сетей. В данном фрагменте мы с вами рассмотрим основные идеи и понятия, используемые в данном продукте, и разберемся с тем, как эта библиотека облегчает построение и тренировку сетей.

В первую очередь, TensorFlow – это пакет, предназначенный для упрощения разработки сетей, проверки гипотез, тренировки моделей, а также для исследовательских целей. Аппарат нейронных сетей сочетает в себе множество «строительных блоков» – разных видов слоев и элементов, которые можно комбинировать друг с другом. Данные блоки являются более-менее стандартизованными и нет смысла каждый раз писать их заново с нуля. TensorFlow представляет собой библиотеку таких блоков вкупе со средствами их обу-

чения, что экономит время на разработку моделей и снижает вероятность ошибок при написании кода.

Вторая важная деталь, присущая многим фреймворкам глубокого обучения, и TensorFlow в том числе – наличие так называемого autograd'a. Вы уже познакомились с тем, как вычислять градиент для некоторых типов блоков в архитектурах нейронных сетей. В дальнейшем мы с вами будем изучать еще множество разных блоков и каждый из них должен быть дифференцируем для корректного обучения с помощью градиентного спуска. Опять же, реализовывать это самостоятельно каждый раз при создании новой модели было бы очень утомительно, да и этот процесс можно автоматизировать. Эта самая автоматизация называется autograd или автоматическое дифференцирование. При создании нейронной сети средствами TensorFlow все блоки вашей сети могут быть автоматически связаны друг с другом и можно определить, каким образом будут высчитываться градиенты, а также куда они будут течь.

В третьих, TensorFlow – это очень эффективно написанная библиотека. Данный фреймворк написан с учетом возможности выполнения на разных процессорах, с разными доступными наборами инструкций, а также с учетом возможности выполнения на графических ускорителях или даже специализированных тензорных ускорителях. Кроме того, внутри нее скрыто огромное количество специализированного кода для того, чтобы максимально эффективно обрабатывать данные. Фреймворк предоставляет API для работы с ним из разных языков программирования, таких как C++, Python, Java, а также существует написанная сообществом поддержка многих других языков. Таким образом, ваш проект можно будет виртуально разделить на две части – «внешний» язык программирования, на котором вы задаете описание вашей модели и который взаимодействует с внешним API фреймворка, и "внутренняя часть" фреймворка, которая будет непосредственно создавать модель и транслировать ваши команды в необходимый код для ее обучения или использования. Это позволяет вам использовать практически любой язык программирования, который вы хотите, и при этом почти не терять в скорости тренировки и использования модели.

Теперь поговорим про **внутреннее устройство фреймворка**. Давайте вкратце рассмотрим, как устроен данный фреймворк и какими понятиями он оперирует. В первую очередь стоит разобраться с тем, что такое «тензор». В терминах фреймворка, тензор – это многомерный массив данных, доступный для обработки и оперирования им. Данный массив определяется одним из доступных типов данных (например, float32), и размерностью. В рамках фреймворка все тензоры иммутабельны – в случае изменения данных создается новый тензор. Можно также оперировать так называемыми «зубчатыми массивами» (они же – jagged array) – массивы, у которых не все элементы в

рамках одной размерности обладают одинаковой длиной, или разреженными массивами.

Переменная в рамках фреймворка TensorFlow – это объект для хранения тензоров, значения которых должны меняться. Обычно переменные используются для хранения, например, параметров модели – весов, которые будут изменяться в процессе обучения.

Также в рамках данного фреймворка часто оперируют понятием «eager execution», оно же – «моментальное исполнение». Взаимодействовать с TensorFlow можно в двух режимах. Первый режим – это режим моментального исполнения, когда все ваши команды и взаимодействия тензоров будут тут же исполнены, и тут же будет посчитан результат. Второй способ – это, так называемый способ задания графа вычислений, когда вы определяете последовательность вычислений над тензорами и переменными, а затем отдельной инструкцией заполняете необходимые тензоры и запускаете процесс вычисления графа. Чаще всего в процессе разработки нейронных сетей используется второй режим, а первый позволяет вам исследовать структуру сети, выполнять код и анализировать вывод на наличие ошибок, то есть заниматься отладкой. В дальнейшем, обсуждая TensorFlow и фреймворк Keras, чаще всего мы будем работать в режиме определения графа и последующего запуска вычислений.

Любые операции, совершаемые над заданными тензорами в рамках фреймворка организуются в граф вычислений. Данный граф описывает тензоры и операции, которые над ними применяются. Этот граф – это полное описание вычислений, которые будут произведены при запуске соответствующих команд и предоставления данных «входным» тензорам – специальным тензорам под названием placeholder. Один раз построив такой граф вычислений и описав им, например, нейронную сеть, можно сериализовать его вместе с весами, обученными в процессе тренировки, и в дальнейшем передавать и использовать.

Фреймворк TensorFlow изначально создавался для общих целей, то есть как для обучения и тренировки простых моделей, так и для исследовательских задач и построения сложных архитектур. Вследствие этого, его структура модулей и функций построена в достаточно общем стиле и для использования в рамках простых задач приходится писать множество повторяющихся вещей, таких как функция тренировки, предоставление коллбэков и так далее. Для упрощения работы мы с вами будем использовать часть TensorFlow под названием Keras – специальный модуль, создающий абстракции более высокого уровня, но дающий меньшую гибкость при построении и использовании модели. Чаще всего, если вам необходимо выполнить инженерную задачу, возможностей модуля Keras вам будет достаточно. Однако, при выполнении исследовательских задач или построении очень сложных архитектур,



вам, возможно, придется обратиться к изначальным компонентам фреймворка TensorFlow.

1.4 Keras

go to ipynb

1.5 Hyperparams

go to ipynb

1.6 Under- and Over-fitting

go to ipynb

2 Оптимизаторы в НС

2.1 Введение

В этой части лекции мы остановимся подробнее на тех популярных алгоритмах оптимизации, которые на данный момент повсеместно используются в нейронных сетях. Наверное, нет нужды мотивировать важность выбора корректного алгоритма оптимизации, ведь, по сути, само обучение НС – это оптимизация. Каждый раз, как только очередной батч данных проходит через нейронную сеть, а та, в свою очередь, выдает предсказания для этого батча, приходится отвечать на следующий вопрос: как использовать информацию о различиях между тем, что выдала нейронная сеть, и известными, истинными значениями предсказанных переменных, для последующего изменения весов модели с целью построения более качественных предсказаний в будущем? Этим вопросом как раз-таки и занимается алгоритм оптимизации, минимизирующий рассматриваемую функцию потерь. Ну что, давайте поговорим про некоторые популярные алгоритмы, а также сравним их между собой.

2.2 SGD и скорость обучения

SGD (stochastic gradient descent, `keras.optimizers.SGD`) – стохастический градиентный спуск – классический алгоритм оптимизации. В принципе, о нем мы уже говорили ранее. Он аналогичен градиентному спуску, но на каждом шаге использует не весь набор тренировочных данных для составления функции потерь, и, соответственно, вычисления градиента, а лишь некоторый батч. Важным параметром является как размер батча, так и шаг, или, так называемый, `learning rate`.

Скорость обучения – это чрезвычайно важный параметр. Если скорость будет слишком большой, то, скорее всего, алгоритм будет «прыгать» по случайным точкам пространства и, тем самым, достигнет минимум с весьма небольшой вероятностью. Если же скорость обучения будет маленькой, то описанная проблема не возникает, но возникает другая, не менее интересная проблема: алгоритм останавливается чуть ли не в первом попавшемся локальном минимуме, значение в котором может быть сильно больше, чем значение в «более хорошем» локальном минимуме, не говоря уже о глобальном, рисунок 1.

Кажется интуитивно понятным, что скорость обучения сначала должна быть достаточно большой, чтобы быстро прийти в нужную область пространства, а затем стать сильно меньше, чтобы детально исследовать окрестность точки минимума и, в конце концов, не пропустить его (и попасть в него). Поэтому простейшая стратегия управления скоростью обучения примерно так



img/step_choice.png

Рис. 1: Выбор шага

и выглядит: начинаем с большой скорости, и постепенно эту скорость уменьшаем. Часто в этом случае используется или линейное затухание

$$\eta = \eta_0 \left(1 - \frac{t}{T}\right)$$

или экспоненциальное

$$\eta = \eta_0 e^{-\frac{t}{T}},$$

где t – это прошедшее с начала обучения время (например, число мини-батчей или число эпох), а T – отвечающий за скорость уменьшения η параметр. Впрочем, мы вряд ли сильно упростили себе задачу – теперь вместо одного параметра η нам надо подбирать два: η_0, T :) Но дело даже не в этом. Наверное, более глобально проблема заключается в следующем: такой подход никак не отражает характер и форму функции, минимум которой мы ищем. Например, если минимум находится в вытянутом «овраге», шаг спуска будет направлен от одной стенки оврага к другой. Когда же точка окажется на второй стенке, градиент опять будет направлен к противоположной стенке, и так далее. В итоге, к минимуму мы будем двигаться очень и очень долго. Попробуем исправить это недоразумение.

2.3 Моментум (Momentum), импульс

Итак, метод импульсов (`keras.optimizers.SGD(momentum=0.01, nesterov=True)`) позволяет ускорить градиентный спуск в нужном (вещающем минимизацию) направлении и уменьшает его, спуска, колебания. С точки зрения физики все достаточно понятно: если считать, что в многомерном пространстве действуют те же законы ньютоновской механики, что и в нашей обыденности, то условный шарик, имеющий некоторую массу, будет быстро набирать скорость, скатываясь по крутому участку вниз к пологой части. Кроме того, шарик обладает инерцией, и его момент движения нельзя изменить мгновенно: ускорение можно направить по градиенту, но вот сама точка какое-то время будет двигаться в том направлении, в котором и двигалась. Метод моментов (импульсов) и моделирует подобную ситуацию. Формально все очень просто:

$$v_t = \gamma v_{t-1} - \eta \nabla L(\omega), \quad \omega = \omega + v_t,$$

где γ – параметр, меньший единицы (часто больший, чем 0.9), отвечающей за то, какую часть прошлого градиента мы готовы взять, чтобы скомбинировать его с текущим, L – функция потерь, которую мы и минимизируем.

Продолжая аналогию с катящимся с горы шариком, полезно не просто катиться вниз с бешеной скоростью, но и посматривать под ноги, чтобы не споткнуться об камень. Это означает, что градиент-то надо вычислять не в точке ω , а в точке $\omega - \gamma v_{t-1}$, то есть в точке, куда мы придем после применения момента предыдущего шага:

$$v_t = \gamma v_{t-1} - \eta \nabla_{\omega} L(\omega - \gamma v_{t-1}).$$

Это – так называемый метод Нестерова.

2.4 Метод Ньютона

Следующая проблема градиентного спуска в многомерных пространствах – это седловые точки. Напомним, что точка называется седловой, если градиент в ней равен нулю, но точка – не точка минимума или максимума. Иными словами, по некоторым направлениям в этой точке функция минимизируется, а по некоторым – максимизируется, рисунок 2. Такие точки встречаются сплошь и рядом, но они нам совершенно неинтересны, и даже вредны, по понятным причинам. На наше счастье, есть несколько успешных модификаций градиентного спуска, которые позволяют немного отойти от седловой точки и успешно «скатиться вниз» по тем направлениям, которые максимизируют данную функцию в заданной точке. Градиент – это линейное приближение, и оно не отслеживает (и не может отследить) чашечкообразную форму функции в окрестности минимума. Одно из возможных решений – это



Рис. 2: Седловая точка

искать не первую производную, а вторую – так называемый метод Ньютона. В основе метода Ньютона лежит идея приближения функции квадратичной функцией (это просто формула Тейлора, не более):

$$L(\omega) \approx L(\omega_0) + \nabla_{\omega} L(\omega_0)(\omega - \omega_0) + \frac{1}{2}(\omega - \omega_0)^T H(\omega)(\omega - \omega_0),$$

где $H(\omega)$ – гессиан $L(\omega)$. Впрочем, этот метод второго порядка требует очень много вычислений и памяти, поэтому в теории нейронных сетей используется редко. У него есть неплохие рабочие аналоги первого порядка.

2.5 Adagrad и Adadelata

Концепция метода Adagrad (`keras.optimizers.adagrad`) предлагает использовать не один и тот же константный learning rate для всех измерений на конкретном шаге, а менять его адаптивно для разных измерений. Пусть

$$g_{t,i} = \nabla_{\omega_i} L(\omega)$$

– градиент функции ошибки по параметру ω_i . Тогда обновление параметра θ_i будем производить по следующему правилу:

$$\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \varepsilon}} g_{t,i},$$

где G_t – диагональная матрица, каждый диагональный элемент которой – сумма квадратов градиентов по соответствующему параметру за все предыдущие шаги, то есть

$$G_{t,ii} = G_{t-1,ii} + g_{t,i}^2,$$

а параметр $\varepsilon > 0$ помогает избежать деления на ноль. В итоге, идейно, мы искусственно повышаем интерес к направлениям, в которых изменение мало, и искусственно понижаем интерес к направлениям, в которых изменение велико. Как результат, мы пытаемся как-то выравнивать интерес ко всем направлениям.

Интересно заметить, что если убрать корень из знаменателя, то алгоритм будет работать куда хуже. Векторно все выглядит так:

$$v_t = -\frac{\eta}{\sqrt{G_{t-1} + \varepsilon}} g_{t-1}.$$

Итак, плюс заключается в том, что теперь learning rate обновляется автоматически на каждом шаге, и об изменении шага можно сильно не задумываться. Есть и минус. Скорость обучения порой уменьшается слишком стремительно (ведь G постоянно увеличивается), а это плохо сказывается на обучении глубоких нейронных сетей. Adadelta (`keras.optimizers.adadelta`) пытается исправить эти недочеты, мы не будем подробно останавливаться на деталях.

2.6 RMSProp

RMSProp (`keras.optimizers.rmsprop`) – метод, похожий на предыдущие. RMS – root mean squares – среднеквадратическое отклонение. По сути, этот метод не накапливает сумму градиентов, как это делает Adagrad, а вычисляет что-то вроде скользящего среднего этих градиентов. Тем самым, в некотором смысле запрещается неограниченный рост суммы градиентов, и, как следствие, неограниченное падение learning rate.

2.7 Adam

Последний метод, о котором мы поговорим – это так называемый Adam (`keras.optimizers.adam`). В некотором смысле можно считать, что этот метод – симбиоз методов RMSProp и Momentum. У этого алгоритма все также есть адаптивный learning rate, но теперь он его применяет не просто к градиенту, а, в некотором смысле, к скорости. Данный алгоритм на данный момент

является одним из самых популярных. Оптимизаторы Adam и RMSProp аппроксимируют вторую производную в точке, что приближает их к методам, вычисляющим вторую производную. Как и этот класс методов, указанные оптимизаторы таким образом оптимизируют процесс обучения в связи с более корректным определением дистанции спуска для каждого из измерений в многомерном пространстве, что обычно выражается в большей вычислительной сложности, но меньше числе итераций.

2.8 Практические советы

Естественно, такое разнообразие алгоритмов (и описаны они далеко не все) приводит к естественному вопросу: а какой же алгоритм выбрать для конкретной задачи? Этот вопрос не имеет какого-то глобального ответа, на практике нужно пробовать разные варианты. Впрочем, если данные достаточно разрежены, то адаптивные алгоритмы неплохо справляются со своей задачей.

Важно отметить и следующее замечание. На практике при использовании спуска важно отслеживать ошибку на валидационном множестве и останавливать процесс в тот момент, когда ошибка начинает расти. Иначе – не избежать оверфиттинга, ведь можно найти достаточно глубокий минимум функции ошибки, который плохо обобщается на новые данные (плохая генерализующая способность). Это и понятно: алгоритмы минимизации не в курсе, что там с обобщением. Они заточены под одну задачу: минимизируй, минимизируй, минимизируй. так что их порывы неплохо бы контролировать.

Кстати, приведенные рассуждения еще раз выступают в пользу деления набора данных на три части: тренировочную, валидационную и тестовую. Ведь, если использовать описанный философский подход, валидационное множество уже используется для обучения модели и, в некотором смысле, превращается в тренировочное.